

SKILL 23: PATH VARIABLE AND COMMAND RESOLUTION

Name: S Rikhil

Roll No: 2520030480

Sec: 8

PART 1: RETRIEVING PATH VARIABLE AND LOCATING EXECUTABLES

Aim

To write a C program to retrieve the PATH environment variable, search its directories, locate executable files, verify permissions, and handle commands that are not found.

Theory

The PATH environment variable contains a list of directories where the operating system searches for executable programs.

The directories are separated by a colon : in Linux.

For example:

```
/usr/local/bin:/usr/bin:/bin
```

When a command such as ls is entered, the system searches these directories to find

the executable file.

The `getenv()` function is used to retrieve the value of the `PATH` variable. The `strtok()`

function can be used to split the `PATH` into individual directories. The `access()` function

checks whether a file exists and whether it has execute permission.

Objectives

1. Retrieve the `PATH` environment variable.
2. Read the command from the user.
3. Separate `PATH` into individual directories.
4. Search each directory.
5. Construct the complete executable path.
6. Check whether the executable exists.
7. Verify execute permission.
8. Display the location of the command.
9. Handle commands that are not found.

Functions Used

Function	Purpose
<code>getenv()</code>	Gets the value of an environment variable
<code>strtok()</code>	Splits PATH into directories
<code>strcpy()</code>	Copies strings
<code>strcat()</code>	Joins strings
<code>access()</code>	Checks file existence and permissions
<code>printf()</code>	Displays output

C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main()
{
    char *path = getenv("PATH");
    char *dir;
    char file[500];
    char cmd[100];

    printf("Enter command: ");
    scanf("%s", cmd);

    dir = strtok(path, ":");
    while (dir != NULL)
    {
        strcpy(file, dir);
```

```
strcat(file, "/");
strcat(file, cmd);

if (access(file, X_OK) == 0)
{
printf("Executable found: %s\n", file);
return 0;
}

dir = strtok(NULL, ":");
}

printf("Command not found\n");

return 0;
}
```

Explanation of Code

1. Retrieve PATH

```
getenv("PATH")
```

Gets the PATH environment variable from the operating

system. **2. Read Command**

```
scanf("%s", cmd);
```

Reads the command entered by the user.

3. Split PATH

```
strtok(path, ":")
```

Separates PATH into individual directories.

4. Create File Path

```
strcpy(file, dir);  
strcat(file, "/");  
strcat(file, cmd);
```

Combines the directory and command

name. Example:

```
/usr/bin + / + ls
```

becomes:

```
/usr/bin/ls
```

5. Check Permission

```
access(file, X_OK)
```

Checks whether the file exists and has execute

permission. **6. Handle Missing Command**

If no executable is found in any PATH

directory: Command not found

is displayed.

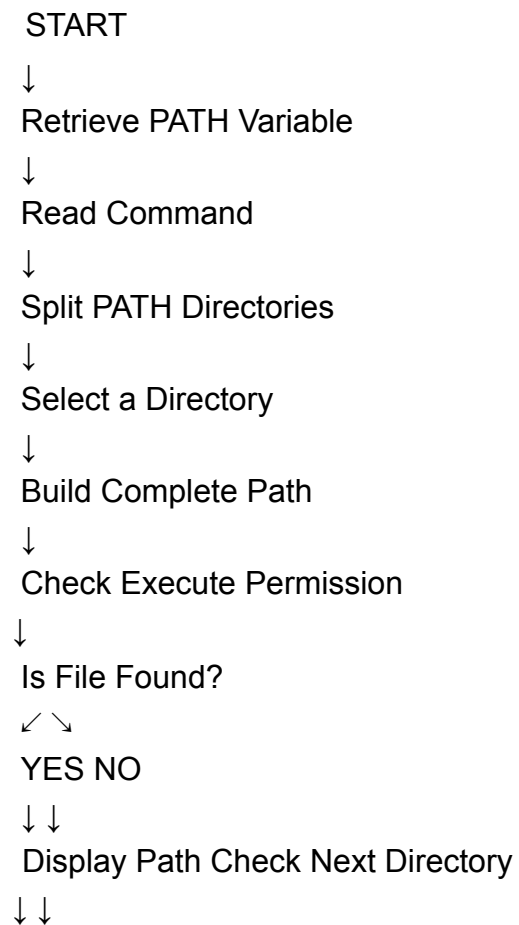
Compile

```
gcc path.c -o path
```

Run

```
./path
```

Flow Chart



STOP More Directories?

↙ ↘

YES NO

↓ ↓

Search Again Display

"Command

not found"

↓

STOP

Sample Output – Command Found

Enter command: ls

Executable found: /usr/bin/ls

Sample Output – Command Not Found

Enter command: abcxyz

Command not found

Observation

Operation	Observation
<code>getenv()</code>	Retrieves PATH
<code>strtok()</code>	Separates PATH directories
<code>strcpy()</code>	Copies directory path
<code>strcat()</code>	Adds command to path
<code>access()</code>	Checks executable permission
Valid command	Executable path is displayed
Invalid command	"Command not found" is displayed

Test Cases

Input	Expected Result
<code>ls</code>	Executable path is displayed
<code>pwd</code>	Executable path is displayed
<code>cat</code>	Executable path is displayed
<code>echo</code>	Executable path is displayed
<code>abcxyz</code>	Command not found

Output

```

EOF
[gcc path.c -o path && ./path
Enter command: ls
Executable found: /bin/ls
7rainreddy7@Gunapatis-MacBook-Air ~ % >....
    dir = strtok(path, ":");

    while (dir != NULL)
    {
        strcpy(file, dir);
        strcat(file, "/");
        strcat(file, cmd);

        if (access(file, X_OK) == 0)
        {
            printf("Executable found: %s\n", file);
            return 0;
        }

        dir = strtok(NULL, ":");
    }

    printf("Command not found\n");

    return 0;
}
EOF
[gcc path.c -o path && ./path
Enter command: abcdxyz
Command not found
7rainreddy7@Gunapatis-MacBook-Air ~ % █

```

Enter command: pwd

Executable found: /usr/bin/pwd

Result

The C program successfully retrieved the PATH environment variable, parsed its directories, searched for the requested command, verified executable permission, and handled missing commands. Thus, command resolution using the PATH variable was successfully demonstrated.